ΜΑΘΗΜΑ 3 ΑΠΟ 15 · ΥΠΟΕΝΟΤΗΤΑ 2.3.Γ.2

Λίστες και πλειάδες

Πολλές τιμές σε μία μεταβλητή

Θεωρία + 3 ασκήσεις + 3 προγράμματα με λύσεις Διάρκεια: 4 ώρες (3η–6η) 26 διαφάνειες

Προγραμματισμός Εφαρμογών Διαδικτύου & Παιχνιδιών (Python, Pygame) · Γ΄ εξάμηνο · Δευτέρα 19/10/2026

Τι θα ξέρεις στο τέλος

1. Να φτιάχνεις λίστες και να παίρνεις στοιχεία με **δείκτες** (και αρνητικούς) και **slicing**.
2. Να προσθέτεις, να αφαιρείς και να ταξινομείς στοιχεία με τις **μεθόδους** της λίστας.
3. Να διατρέχεις λίστα με **for** και να υπολογίζεις άθροισμα, μέσο όρο, μέγιστο, πλήθος.
4. Να φτιάχνεις νέα λίστα από παλιά σε μία γραμμή με **list comprehension**.
5. Να δουλεύεις **πίνακα 2 διαστάσεων** (λίστα από λίστες).
6. Να ξέρεις πότε θέλεις **πλειάδα** (tuple) αντί για λίστα — και γιατί θα τη χρειαστεί το pygame.

Πώς είναι οργανωμένο το 4ωρο

Ώρες 1–2: λίστες — δημιουργία, δείκτες, μέθοδοι, διάσχιση — και πλειάδες. **Ώρες 3–4:** εργαστήριο — βαθμολογίες τάξης, παράλληλες λίστες και στο τέλος ένα **μαγικό τετράγωνο 3×3**.

ΣΎΝΔΕΣΗ Το for της προηγούμενης φοράς διέτρεχε range και κείμενα. Σήμερα θα διατρέχει λίστες — εκεί δείχνει όλη του τη δύναμη.

Θυμάσαι από την προηγούμενη φορά;

ΕΚΦΏΝΗΣΗ

Πέντε γρήγορες ερωτήσεις, 5 λεπτά. Τι εμφανίζει κάθε κομμάτι;

ΛΎΣΗ

ΚΩΔΙΚΑΣ	ΑΠΟΤΈΛΕΣΜΑ	ΓΙΑΤΊ
`for i in range(2, 6): print(i)`	`2 3 4 5`	χωρίς το τέλος
`for ch in "ok": print(ch)`	`o k`	γράμμα-γράμμα
`len("Python")`	`6`	μήκος κειμένου
`"Py"[0]`	`"P"`	πρώτο γράμμα
`"y" in "Python"`	`True`	υπάρχει μέσα

Γιατί αυτά

Ό,τι ισχύει για τα κείμενα — **len**, **[0]**, **in**, **for** — ισχύει σχεδόν αυτούσιο και για τις λίστες. Ξεκινάμε με πλεονέκτημα.

ΠΑΡΑΤΉΡΗΣΗ Ένα κείμενο είναι ήδη μια «σειρά από γράμματα». Η λίστα είναι μια σειρά από **οτιδήποτε**.

30 βαθμοί, 30 μεταβλητές;

ΧΩΡΊΣ ΛΊΣΤΑ

```
grade1 = 12
grade2 = 8.5
grade3 = 17
# ... άλλες 27 γραμμές ...
avg = (grade1 + grade2 + grade3
       + ...) / 30
```

ΜΕ ΛΊΣΤΑ

```
grades = [12, 8.5, 17, 9, 15]

avg = sum(grades) / len(grades)
print(f"{avg:.2f}")
12.30
```

Τι πάει στραβά

Ο κώδικας **μεγαλώνει** με το πλήθος. Και αν τους βαθμούς τους δίνει ο χρήστης, **δεν ξέρεις** πόσες μεταβλητές να φτιάξεις.

Τι κερδίζεις

Ένα όνομα για όλες τις τιμές. Ο ίδιος κώδικας δουλεύει για 5, 30 ή 3.000 βαθμούς — χωρίς καμία αλλαγή.

Η ΙΔΈΑ Η λίστα είναι για τις τιμές ό,τι ο βρόχος για τις εντολές: γράφεις μία φορά, δουλεύει για όσα χρειαστεί.

Φτιάχνουμε λίστες

```
empty = []
grades = [12, 8.5, 17]
names = ["Άννα", "Νίκος"]
mixed = [7, "Python", True, 3.14]

print(len(grades))
3
print(list(range(5)))
[0, 1, 2, 3, 4]
print(list("abc"))
['a', 'b', 'c']
```

Η σύνταξη

Αγκύλες **[ ]**, στοιχεία χωρισμένα με **κόμμα**. Η κενή λίστα είναι **[]**.

Μεγαλώνει και μικραίνει

Δεν δηλώνεις μέγεθος από πριν. Η λίστα **προσαρμόζεται** καθώς προσθέτεις ή αφαιρείς στοιχεία — σε αντίθεση με τους πίνακες σταθερού μεγέθους άλλων γλωσσών.

Οποιοσδήποτε τύπος

Μπορεί να έχει και **διαφορετικούς τύπους** μαζί. Στην πράξη, κράτα μία λίστα για ένα είδος δεδομένων.

ΧΡΉΣΙΜΟ Το **list()** μετατρέπει σε λίστα ό,τι διατρέχεται: ένα range, ένα κείμενο — και αργότερα αρχεία.

Κάθε στοιχείο έχει θέση

fruits = ["μήλο", "αχλάδι", "κεράσι", "σύκο"]

ΘΈΣΗ	0	1	2	3
τιμή	"μήλο"	"αχλάδι"	"κεράσι"	"σύκο"
από το τέλος	-4	-3	-2	-1

```
>>> fruits[0]
'μήλο'
>>> fruits[-1]
'σύκο'
>>> fruits[4]
IndexError: list index out of range
```

Το μέτρημα ξεκινά από το 0

Το **πρώτο** στοιχείο είναι το **[0]**, το τελευταίο το **[len − 1]**. Με 4 στοιχεία, το [4] δεν υπάρχει.

Αρνητικοί δείκτες

Το **[-1]** είναι πάντα το τελευταίο — όσο μεγάλη κι αν είναι η λίστα.

ΤΟ ΠΙΟ ΣΥΧΝΌ ΣΦΆΛΜΑ **IndexError**: ζήτησες θέση που δεν υπάρχει. Σχεδόν πάντα είναι το «ένα παραπάνω» — το [len] αντί για το [len − 1].

Η λίστα αλλάζει, το κείμενο όχι

ΛΊΣΤΑ · ΑΛΛΆΖΕΙ

```
grades = [12, 8.5, 17]
grades[1] = 10
print(grades)
[12, 10, 17]
```

ΚΕΊΜΕΝΟ · ΔΕΝ ΑΛΛΆΖΕΙ

```
word = "Γάτα"
word[0] = "Μ"
TypeError: 'str' object does not
support item assignment
```

Μεταβλητή (mutable)

Μπορείς να αλλάξεις **οποιοδήποτε στοιχείο** της λίστας, χωρίς να φτιάξεις καινούργια.

Αμετάβλητο (immutable)

Το κείμενο δεν αλλάζει. Φτιάχνεις **νέο**: **word = "Μ" + word[1:]** → "Μάτα".

Γιατί μας νοιάζει

Η διάκριση «αλλάζει / δεν αλλάζει» είναι όλη η διαφορά ανάμεσα σε **λίστα** και **πλειάδα**, που βλέπουμε σε λίγο.

ΌΡΟΣ Στα βιβλία θα το βρεις ως **mutable** (μεταβλητό) και **immutable** (αμετάβλητο).

Κομμάτι λίστας: [αρχή:τέλος]

nums = [10, 20, 30, 40, 50, 60]

ΓΡΆΦΕΙΣ	ΠΑΊΡΝΕΙΣ	ΣΗΜΑΊΝΕΙ
`nums[1:4]`	`[20, 30, 40]`	θέσεις 1, 2, 3
`nums[:3]`	`[10, 20, 30]`	τα 3 πρώτα
`nums[3:]`	`[40, 50, 60]`	από τη θέση 3 ως το τέλος
`nums[-2:]`	`[50, 60]`	τα 2 τελευταία
`nums[::2]`	`[10, 30, 50]`	ανά δύο
`nums[::-1]`	`[60, 50, ..., 10]`	ανάποδα

Ίδιος κανόνας με το range

Το **τέλος δεν περιλαμβάνεται**. Το [1:4] δίνει 4 − 1 = **3** στοιχεία.

Δεν «σκάει»

Το slicing εκτός ορίων **δεν δίνει σφάλμα**: το nums[4:100] δίνει απλώς [50, 60].

ΙΣΧΎΕΙ ΚΑΙ ΓΙΑ ΚΕΊΜΕΝΑ Το **"Python"[::-1]** δίνει **"nohtyP"**. Ο έλεγχος παλίνδρομου γίνεται σε μία γραμμή: word == word[::-1].

Δείκτες και κομμάτια

ΕΚΦΏΝΗΣΗ

Με τη λίστα δίπλα, βρες τι δίνει κάθε έκφραση. Πρώτα στο χαρτί, μετά στη γραμμή εντολών.

```
days = ["Δευ", "Τρι", "Τετ", "Πεμ", "Παρ"]
```

ΛΎΣΗ

ΈΚΦΡΑΣΗ	ΑΠΟΤΈΛΕΣΜΑ	ΣΧΌΛΙΟ
`days[1]`	`"Τρι"`	δεύτερη θέση
`days[-2]`	`"Πεμ"`	προτελευταίο
`days[1:3]`	`["Τρι", "Τετ"]`	χωρίς τη θέση 3
`days[3:]`	`["Πεμ", "Παρ"]`	ως το τέλος
`len(days[:2])`	`2`	λίστα με 2 στοιχεία
`days[5]`	**IndexError**	θέσεις 0 έως 4
`days[0][0]`	`"Δ"`	πρώτο γράμμα του πρώτου

Η πιο ύπουλη

Το **days[0][0]**: το days[0] είναι το κείμενο "Δευ", και το [0] του κειμένου είναι το "Δ". Οι δείκτες **στοιβάζονται**.

ΣΤΌΧΟΣ Να «βλέπεις» τη λίστα με τις θέσεις της. Όταν διστάζεις, γράψε τους δείκτες από κάτω, όπως στη διαφάνεια 6.

Προσθήκη στοιχείων

```
cart = ["ψωμί"]

cart.append("γάλα")        # στο τέλος
cart.insert(0, "αυγά")     # σε θέση
print(cart)
['αυγά', 'ψωμί', 'γάλα']

cart = cart + ["τυρί", "μέλι"]
print(len(cart))
5
```

ΜΈΘΟΔΟΣ	ΤΙ ΚΆΝΕΙ
`.append(x)`	βάζει το x **στο τέλος**
`.insert(i, x)`	βάζει το x **στη θέση i**
`.extend(list)`	βάζει **όλα** τα στοιχεία άλλης λίστας
`list1 + list2`	φτιάχνει **νέα** ενωμένη λίστα

Η «τελεία»

Οι μέθοδοι καλούνται **πάνω στη λίστα**: lista.append(...). Αλλάζουν την ίδια τη λίστα.

Η παγίδα append ή extend

Το **cart.append(["τυρί", "μέλι"])** βάζει **μία** λίστα μέσα στη λίστα. Για να μπουν τα δύο στοιχεία χωριστά, θέλεις **extend**.

ΣΤΗΝ ΠΡΆΞΗ Το **append** μέσα σε βρόχο είναι ο νούμερο ένα τρόπος να γεμίσεις μια λίστα από τον χρήστη. Θα τον δεις στο εργαστήριο.

Αφαίρεση στοιχείων

```
nums = [5, 3, 8, 3, 1]

nums.remove(3)    # το πρώτο 3
[5, 8, 3, 1]
last = nums.pop()  # το τελευταίο
last = 1,  nums = [5, 8, 3]
first = nums.pop(0)
first = 5, nums = [8, 3]
del nums[0]
[3]
```

ΕΝΤΟΛΉ	ΑΦΑΙΡΕΊ	ΕΠΙΣΤΡΈΦΕΙ
`.remove(x)`	την **τιμή** x	τίποτα
`.pop()`	το τελευταίο	**το στοιχείο**
`.pop(i)`	τη **θέση** i	**το στοιχείο**
`del lst[i]`	τη θέση i	τίποτα

Αν δεν υπάρχει

Το **remove(99)** σε λίστα χωρίς 99 δίνει **ValueError**. Έλεγξε πρώτα: **if 99 in nums: nums.remove(99)**.

remove: τιμή, pop: θέση

Το **remove(3)** ψάχνει την **τιμή** 3. Το **pop(3)** βγάζει ό,τι βρίσκεται στη **θέση** 3. Μπερδεύονται συχνά.

ΠΌΤΕ POP Όταν θέλεις και να βγάλεις και να **χρησιμοποιήσεις** το στοιχείο: π.χ. «τράβα την επόμενη κάρτα από την τράπουλα».

Ερωτήσεις σε μια λίστα

grades = [12, 8.5, 17, 9, 17, 15]

ΓΡΆΦΕΙΣ	ΑΠΟΤΈΛΕΣΜΑ	ΑΠΑΝΤΆ ΣΤΟ
`len(grades)`	`6`	πόσα στοιχεία;
`sum(grades)`	`78.5`	άθροισμα
`min(grades)`	`8.5`	ο μικρότερος
`max(grades)`	`17`	ο μεγαλύτερος
`17 in grades`	`True`	υπάρχει;
`grades.count(17)`	`2`	πόσες φορές;
`grades.index(17)`	`2`	σε ποια θέση (πρώτη);

Μέσος όρος

Δεν υπάρχει έτοιμο avg(). Γράφεις **sum(grades) / len(grades)** → **13.08** (με :.2f).

Λίστα χωρίς στοιχεία

Τα **max([])** και **min([])** δίνουν σφάλμα, και ο μέσος όρος διαίρεση με 0. Έλεγξε **if grades:** πριν.

ΣΎΝΔΕΣΗ Το **in** που έλεγχε γράμματα σε κείμενο, εδώ ελέγχει στοιχεία σε λίστα. Ίδια λέξη, ίδια λογική.

sort() ή sorted();

LIST.SORT() · ΑΛΛΆΖΕΙ ΤΗ ΛΊΣΤΑ

```
nums = [5, 2, 9, 1]
nums.sort()
print(nums)
[1, 2, 5, 9]
nums.sort(reverse=True)
[9, 5, 2, 1]
```

SORTED(LIST) · ΦΤΙΆΧΝΕΙ ΝΈΑ

```
nums = [5, 2, 9, 1]
new = sorted(nums)
print(new)
[1, 2, 5, 9]
print(nums)
[5, 2, 9, 1]   # ίδια!
```

Πότε sort()

Όταν δεν σε νοιάζει η αρχική σειρά. Αλλάζει τη λίστα **επί τόπου**.

Πότε sorted()

Όταν θέλεις να **κρατήσεις** και την αρχική σειρά. Δουλεύει και σε κείμενα: **sorted("cab")**.

```
nums = nums.sort()
print(nums)
None
```

Η παγίδα

Το **sort()** δεν επιστρέφει τίποτα (**None**). Μην το εκχωρείς σε μεταβλητή — χάνεις τη λίστα.

ΕΠΊΣΗΣ Το **nums.reverse()** αντιστρέφει τη σειρά χωρίς να ταξινομήσει. Τα κείμενα ταξινομούνται **αλφαβητικά**.

for πάνω σε λίστα

1 · ΣΤΟΙΧΕΊΟ-ΣΤΟΙΧΕΊΟ

```
names = ["Άννα", "Νίκος", "Ελένη"]
for name in names:
    print("Γεια σου,", name)
Γεια σου, Άννα
...
```

2 · ΜΕ ΘΈΣΗ (ENUMERATE)

```
for i, name in enumerate(names, start=1):
    print(f"{i}. {name}")
1. Άννα
2. Νίκος
3. Ελένη
```

3 · ΦΤΙΆΧΝΩ ΝΈΑ ΛΊΣΤΑ ΜΈΣΑ ΣΕ ΒΡΌΧΟ

```
prices = [10, 25, 40]
with_vat = []
for p in prices:
    with_vat.append(round(p * 1.24, 2))
print(with_vat)
[12.4, 31.0, 49.6]
```

Το μοτίβο «κενή λίστα + append»

Ξεκινάς με **[]** έξω από τον βρόχο και προσθέτεις μέσα. Ίδια λογική με τον αθροιστή που ξεκινά από 0.

Μην αλλάζεις τη λίστα που διατρέχεις

Αφαίρεση μέσα σε for πάνω στην ίδια λίστα πηδάει στοιχεία. Φτιάξε **νέα**.

ΚΑΙ ΜΕ RANGE; Το **for i in range(len(names))** δουλεύει, αλλά το **enumerate** είναι πιο καθαρό όταν θέλεις και θέση και τιμή.

Βήμα-βήμα: τι έχει η λίστα;

ΕΚΦΏΝΗΣΗ

Γράψε πώς είναι η λίστα **μετά από κάθε γραμμή**. Τι τυπώνει στο τέλος;

```
lst = [3, 1, 2]
lst.append(5)
lst.insert(0, 9)
lst.remove(1)
x = lst.pop()
lst.sort()
print(lst, x)
```

Η παγίδα

Το **remove(1)** βγάζει την **τιμή** 1, όχι το στοιχείο της θέσης 1 (που εκείνη τη στιγμή είναι το 3).

ΛΎΣΗ

ΓΡΑΜΜΉ	LST	X
[3, 1, 2]	[3, 1, 2]	—
append(5)	[3, 1, 2, 5]	—
insert(0, 9)	[9, 3, 1, 2, 5]	—
remove(1)	[9, 3, 2, 5]	—
pop()	[9, 3, 2]	5
sort()	[2, 3, 9]	5

Τυπώνει

[2, 3, 9] 5

ΤΕΧΝΙΚΉ Ο πίνακας τιμών δουλεύει και για λίστες: μια στήλη για κάθε μεταβλητή, μια γραμμή για κάθε εντολή.

Δύο ονόματα, μία λίστα

Η ΠΑΓΊΔΑ

```
a = [1, 2, 3]
b = a
b.append(4)
print(a)
[1, 2, 3, 4]   # άλλαξε κι η a!
```

Η ΣΩΣΤΉ ΑΝΤΙΓΡΑΦΉ

```
a = [1, 2, 3]
b = a.copy()      # ή a[:]
b.append(4)
print(a)
[1, 2, 3]
```

Τι έγινε

Το **b = a** δεν φτιάχνει δεύτερη λίστα. Δίνει **δεύτερο όνομα** στην **ίδια** λίστα. Ό,τι αλλάξεις από το b φαίνεται και από το a.

Η λύση

Το **.copy()** (ή το **a[:]**) φτιάχνει **νέα** λίστα με τα ίδια στοιχεία. Τώρα οι δύο είναι ανεξάρτητες.

Γιατί με τους αριθμούς δεν συμβαίνει

Οι αριθμοί και τα κείμενα **δεν αλλάζουν** (immutable), οπότε το x = y δεν κρύβει εκπλήξεις. Με λίστες, που αλλάζουν, κρύβει.

ΚΑΝΌΝΑΣ Θέλεις αντίγραφο λίστας για να το πειράξεις; Γράψε **.copy()**. Πάντα.

List comprehension

ΜΕ ΒΡΌΧΟ

```
squares = []
for x in range(1, 6):
    squares.append(x * x)
[1, 4, 9, 16, 25]
```

ΣΕ ΜΊΑ ΓΡΑΜΜΉ

```
squares = [x * x for x in range(1, 6)]


[1, 4, 9, 16, 25]
```

Η μορφή

[τι_βάζω for στοιχείο in κάτι if συνθήκη] — το if στο τέλος είναι προαιρετικό και λειτουργεί σαν φίλτρο.

ΚΩΔΙΚΑΣ	ΑΠΟΤΈΛΕΣΜΑ
`[g for g in grades if g >= 10]`	μόνο οι βαθμοί που πέρασαν
`[n.upper() for n in names]`	όλα τα ονόματα με κεφαλαία
`[x for x in range(20) if x % 3 == 0]`	`[0, 3, 6, 9, 12, 15, 18]`

ΜΈΤΡΟ Αν η γραμμή γίνει δυσανάγνωστη, γύρνα στον κανονικό βρόχο. Ο σκοπός είναι η καθαρότητα, όχι η συντομία.

Λίστα από λίστες: πίνακας

```
m = [[1, 2, 3],
     [4, 5, 6],
     [7, 8, 9]]
print(m[1][2])
6
```

```
for row in m:
    for x in row:
        print(x, end=" ")
    print()
1 2 3
4 5 6 ...
```

	ΣΤΉΛΗ 0	ΣΤΉΛΗ 1	ΣΤΉΛΗ 2
γραμμή 0	1	2	3
γραμμή 1	4	5	**6** ← m[1][2]
γραμμή 2	7	8	9

Ένθετοι βρόχοι

Ο εξωτερικός διατρέχει **γραμμές**, ο εσωτερικός τα **στοιχεία** κάθε γραμμής — όπως στην προπαίδεια της προηγούμενης φοράς.

Δύο δείκτες

Πρώτα η **γραμμή**, μετά η **στήλη**: **m[γραμμή][στήλη]**. Το m[1] είναι ολόκληρη η γραμμή [4, 5, 6].

Πού θα το δεις

Σκακιέρα, τρίλιζα, πλέγμα πίστας σε παιχνίδι 2D — όλα πίνακες γραμμών × στηλών.

ΠΛΉΘΗ **len(m)** = πλήθος γραμμών. **len(m[0])** = πλήθος στηλών.

tuple: λίστα που δεν αλλάζει

```
point = (3, 4)
red = (255, 0, 0)
single = (5,)        # προσοχή στο κόμμα

print(point[0])
3
point[0] = 10
TypeError: 'tuple' object does not
support item assignment
```

Το (5,)

Το **(5)** είναι απλώς ο αριθμός 5 σε παρένθεση. Για πλειάδα ενός στοιχείου χρειάζεται **κόμμα**: (5,).

Η σύνταξη

Παρενθέσεις ( ) αντί για αγκύλες. Δείκτες, slicing, len, in, for δουλεύουν **ακριβώς** όπως στη λίστα.

Η διαφορά

Δεν αλλάζει: ούτε append, ούτε remove, ούτε εκχώρηση σε θέση.

Γιατί να τη θέλω;

Για δεδομένα που **ανήκουν μαζί** και δεν πρέπει να αλλάξουν κατά λάθος: συντεταγμένες, ημερομηνία, χρώμα.

PYGAME Στο pygame οι θέσεις είναι πλειάδες **(x, y)** και τα χρώματα **(R, G, B)**: screen.fill((255, 255, 255)).

Unpacking: μία πλειάδα, πολλές μεταβλητές

```
point = (3, 4)
x, y = point
print(x, y)
3 4
```

Ήταν εδώ από την αρχή

Το **a, b = b, a** του 1ου μαθήματος είναι unpacking: δεξιά φτιάχνεται η πλειάδα **(b, a)** και μοιράζεται αριστερά.

```
students = [("Άννα", 17), ("Νίκος", 9),
            ("Ελένη", 15)]

for name, grade in students:
    if grade >= 10:
        print(name, "πέρασε")
Άννα πέρασε  ·  Ελένη πέρασε
```

Λίστα από πλειάδες

Κάθε μαθητής είναι μια πλειάδα **(όνομα, βαθμός)**: τα δύο ανήκουν μαζί. Στο for τα «ξεπακετάρεις» κατευθείαν σε **name** και **grade**.

Πλήθος μεταβλητών = πλήθος στοιχείων

Το **x, y = (1, 2, 3)** δίνει **ValueError: too many values to unpack**. Αριστερά και δεξιά πρέπει να ταιριάζουν.

ΜΠΡΟΣΤΆ Μια συνάρτηση που «επιστρέφει δύο τιμές» στην πραγματικότητα επιστρέφει μία πλειάδα. Θα το δούμε στις συναρτήσεις (9/11).

Λίστα, πλειάδα, κείμενο

	ΛΊΣΤΑ	ΠΛΕΙΆΔΑ	ΚΕΊΜΕΝΟ
Σύνταξη	`[1, 2, 3]`	`(1, 2, 3)`	`"abc"`
Αλλάζει;	ναι	όχι	όχι
Περιέχει	οτιδήποτε	οτιδήποτε	μόνο χαρακτήρες
append, sort, remove	ναι	όχι	όχι
[i], [a:b], len, in, for	ναι	ναι	ναι
Τυπική χρήση	συλλογή που μεγαλώνει	εγγραφή σταθερών πεδίων	κείμενο

Μετατροπές

list((1, 2)) → [1, 2] · **tuple([1, 2])** → (1, 2) · **list("ab")** → ['a', 'b']

Ο γρήγορος κανόνας

Θα **προσθέτεις / αφαιρείς** στοιχεία; → λίστα. Είναι ένα **«πακέτο»** που δεν αλλάζει; → πλειάδα.

Λίστα ή πλειάδα;

ΕΚΦΏΝΗΣΗ

Τι θα διάλεγες; α) Τα προϊόντα στο καλάθι ενός e-shop. β) Οι συντεταγμένες (x, y) ενός παίκτη τη στιγμή που σκοράρει. γ) Ένα χρώμα RGB. δ) Οι βαθμοί που δίνει ο χρήστης ένας-ένας. ε) Οι 7 μέρες της εβδομάδας.

ΛΎΣΗ

`α · λίστα`	Ο πελάτης **προσθέτει και αφαιρεί** προϊόντα → αλλάζει.
`β · πλειάδα`	Τα x και y είναι ένα **«πακέτο»** — μια στιγμή που δεν αλλάζει.
`γ · πλειάδα`	Τρεις τιμές (R, G, B) που ανήκουν **μαζί**. Έτσι τα θέλει και το pygame.
`δ · λίστα`	Γεμίζει **σταδιακά** με append μέσα σε βρόχο.
`ε · πλειάδα`	Σταθερές: **δεν πρέπει** να αλλάξουν κατά λάθος.

ΣΗΜΕΊΩΣΗ Και η λίστα θα «δούλευε» στα β, γ, ε. Η πλειάδα όμως **δηλώνει πρόθεση**: «αυτό δεν αλλάζει» — και το προστατεύει.

Βαθμολογίες τάξης

ΕΚΦΏΝΗΣΗ

Διάβασε το πλήθος των μαθητών και τους βαθμούς τους. Εμφάνισε **μέσο όρο**, **μέγιστο** και **ελάχιστο**, **πόσοι πέρασαν** (≥ 10) και τους βαθμούς **ταξινομημένους** από τον μεγαλύτερο.

ΛΎΣΗ

```
n = int(input("Πόσοι μαθητές; "))
grades = []
for i in range(n):
    g = float(input(f"Βαθμός {i + 1}: "))
    grades.append(g)

avg = sum(grades) / len(grades)
passed = [g for g in grades if g >= 10]

print(f"Μέσος όρος: {avg:.2f}")
print(f"Μέγιστος: {max(grades)}  Ελάχιστος: {min(grades)}")
print(f"Πέρασαν: {len(passed)} από {n}")
print("Ταξινομημένοι:", sorted(grades, reverse=True))
```

```
Πόσοι μαθητές; 5
Βαθμός 1: 12
Βαθμός 2: 8.5
...
Μέσος όρος: 12.30
Μέγιστος: 17.0  Ελάχιστος: 8.5
Πέρασαν: 3 από 5
Ταξινομημένοι: [17.0, 15.0,
  12.0, 9.0, 8.5]
```

Γιατί 17.0 και όχι 17

Διαβάσαμε με **float()**, οπότε όλοι οι βαθμοί είναι float — και τυπώνονται με .0.

ΑΝ ΔΏΣΕΙ 0 Με n = 0 ο μέσος όρος διαιρεί με το μηδέν. Βελτίωση: τύλιξε τους υπολογισμούς σε **if grades:**.

Ποιοι είναι πάνω από τον μέσο όρο;

ΕΚΦΏΝΗΣΗ

Έχεις δύο λίστες: τα **ονόματα** και τους **βαθμούς** (στην ίδια σειρά). Εμφάνισε όσους είναι **πάνω από τον μέσο όρο** και το όνομα του **καλύτερου**.

ΛΎΣΗ

```
names = ["Άννα", "Νίκος", "Μαρία", "Γιώργος", "Ελένη"]
grades = [12, 8.5, 17, 9, 15]

avg = sum(grades) / len(grades)
print(f"Μέσος όρος: {avg:.2f}")

for name, g in zip(names, grades):
    if g > avg:
        print(f"  {name}: {g}")

best = names[grades.index(max(grades))]
print("Καλύτερος/η:", best)
```

```
Μέσος όρος: 12.30
  Μαρία: 17
  Ελένη: 15
Καλύτερος/η: Μαρία
```

zip()

Περπατάει **δύο λίστες μαζί** και δίνει ζευγάρια (όνομα, βαθμός).

Η γραμμή του best

max → **17** · index(17) → **2** · names[2] → **Μαρία**.

ΕΝΑΛΛΑΚΤΙΚΆ Με μία λίστα από πλειάδες (όπως στη διαφάνεια 20) τα δεδομένα κάθε μαθητή μένουν μαζί και δεν κινδυνεύουν να «ξεσυγχρονιστούν».

Είναι μαγικό τετράγωνο;

ΕΚΦΏΝΗΣΗ

Ένας πίνακας 3×3 είναι **μαγικός** όταν όλες οι γραμμές, όλες οι στήλες και οι δύο διαγώνιοι έχουν το **ίδιο άθροισμα**. Υπολόγισε τα αθροίσματα και πες αν είναι μαγικός.

```
m = [[2, 7, 6],
     [9, 5, 1],
     [4, 3, 8]]

row_sums = [sum(row) for row in m]

col_sums = []
for c in range(3):
    total = 0
    for r in range(3):
        total += m[r][c]
    col_sums.append(total)

diag = sum([m[i][i] for i in range(3)])
anti = sum([m[i][2 - i] for i in range(3)])

all_sums = row_sums + col_sums + [diag, anti]
print("Γραμμές:", row_sums, " Στήλες:", col_sums)
print("Διαγώνιοι:", diag, anti)
print("Μαγικό!" if min(all_sums) == max(all_sums) else "Όχι μαγικό")
```

	ΣΤ. 0	ΣΤ. 1	ΣΤ. 2
γρ. 0	2	7	6
γρ. 1	9	**5**	1
γρ. 2	4	3	8

```
Γραμμές: [15, 15, 15]  Στήλες: [15, 15, 15]
Διαγώνιοι: 15 15
Μαγικό!
```

Τα πάντα μαζί

2D λίστα, **ένθετοι βρόχοι**, **comprehension**, **sum / min / max** και ένωση λιστών με **+**.

ΠΡΌΚΛΗΣΗ Άλλαξε ένα στοιχείο (π.χ. το 5 σε 6) και τρέξε ξανά. Μετά κάνε τον κώδικα να δουλεύει για **οποιοδήποτε** N×N με len(m).

Τι κρατάμε

Λίστα [ ]

Αλλάζει. append, insert, remove, pop, sort. Δείκτες από το **0**, το **[-1]** είναι το τελευταίο.

Πλειάδα ( )

Δεν αλλάζει. Για «πακέτα» τιμών: (x, y), (R, G, B). Unpacking: **x, y = point**.

Δύο παγίδες

Το **b = a** δεν αντιγράφει — θέλει **.copy()**. Το **sort()** επιστρέφει **None**.

Με **for** διατρέχεις, με **sum / len / max / min** απαντάς σε ερωτήσεις, με **comprehension** φτιάχνεις νέες λίστες σε μία γραμμή, και με **λίστες από λίστες** φτιάχνεις πίνακες.

Επόμενη φορά · Δευ 26/10

Σύνολα και λεξικά: συλλογές χωρίς διπλότυπα και αναζήτηση με «κλειδί» — π.χ. όνομα → τηλέφωνο.

ΓΙΑ ΤΟ ΣΠΊΤΙ Διάβαζε λέξεις μέχρι να δοθεί κενή γραμμή. Εμφάνισε πόσες είναι, τις λέξεις **αλφαβητικά** και πόσες ξεκινούν από «Α».